Logistics / Delivery Optimization Problem

Abstract:

This document covers the design, methodology, and evaluation of a delivery optimization system. The system reads delivery data from a CSV, sorts by priority and distance, assigns deliveries to three agents with balanced workload, and outputs a structured execution plan per agent.

The system was built iteratively starting from basic sorting and greedy assignment, then improving step by step through Integer Linear Programming, TSP-based route sequencing, fallback handling, validation, and outlier detection.

1. Problem Statement

A warehouse needs to dispatch deliveries to multiple locations daily. Three agents are available. Each delivery has a Location ID, a Distance from the warehouse, and a Priority (High / Medium / Low).

The system must:

1. Read the CSV
2. Sort deliveries by priority and distance
3. Assign deliveries to 3 agents
4. Ensure nearly equal total distance per agent
5. Output a delivery plan with total distance per agent

The core challenge is a trade-off: focusing only on distance ignores urgency. Focusing only on priority creates imbalance. Both must be satisfied simultaneously.

2. Project Evolution:

I've learned new approaches from the limitations of every previous versions.

Version 1 — Sort and Greedy Assign Sorted by priority then distance, assigned each delivery to the least-loaded agent. Satisfied basic requirements but greedy decisions cannot be reversed imbalance of 20–30% on some inputs. High-priority deliveries sometimes clustered on one agent.

Version 2 — Iterative Search Tried shuffling deliveries 500 times, evaluating a cost function, picking the best. Explored combinations rather than one greedy pass. Problem: results changed between runs and there was no convergence guarantee unsuitable for a system that needs consistent, auditable outputs.

Version 3 — Hybrid with Constraints Added penalty logic and constraints to the cost function. More controlled but still relied on weight tuning with no mathematical guarantee. Difficult to explain why specific weights were chosen.

Version 4 — Integer Linear Programming Replaced iterative search with ILP. Considers all assignments simultaneously, finds the provably optimal solution under constraints, and produces the same result every run. A critical discovery here: sorting before ILP does not preserve priority ILP reassigns freely by distance and has no awareness of the sorted order. An explicit constraint was needed to prevent High-priority clustering.

Version 5 — TSP Route Sequencing After ILP decides who gets what, the delivery order within each agent's route still needed solving. Added a nearest-neighbour TSP heuristic applied within each priority group separately not globally. Global TSP would sequence a Low-priority stop first if it is geographically convenient. Group-wise TSP preserves priority order while improving efficiency within each tier.

Version 6 — Batch Processing and Metrics Extended to process multiple CSV files in one run. Added Imbalance Score, Priority Compliance, and a summary report across all inputs.

Version 7 — Robustness Added edge case handling for fewer deliveries than agents, corrected Priority Compliance formula using ceiling division, improved reporting consistency.

Version 8 — Final System Added input validation with case normalisation, greedy fallback if ILP fails, safety net for unassigned deliveries, outlier detection using 2 standard deviations, and Location_ID as sort tie-breaker for full reproducibility.

3. Methodology

Task 1 — Read and Validate CSV

Reads the CSV and immediately checks required columns exist. Priority values are normalised using str.capitalize() so HIGH, high, and High all map correctly. Unrecognised values raise an error naming the exact bad values rather than failing silently. Catching data errors at entry is cheaper than discovering them after dispatch.

Task 2 — Sort by Priority and Distance

Three-key sort: PriorityScore descending → Distance ascending → Location_ID ascending as tie-breaker. The tie-breaker ensures the same input always produces the same sorted output regardless of machine or pandas version. Without it the sort is non-deterministic when two deliveries share identical priority and distance.

Tasks 3 and 4 — ILP Assignment

Objective: Minimise (max_agent_distance − min_agent_distance) This directly targets Task 4 — pushing the solver toward equal distribution.

Five constraints:

1. Each delivery assigned to exactly one agent
2. max_dist ≥ each agent's total (upper bound anchor)
3. min_dist ≤ each agent's total (lower bound anchor)
4. Count cap: no agent receives more than ceil(n/3) deliveries — prevents one agent hoarding many short trips while another gets few long ones
5. High-priority spread: no agent receives more than ceil(n_high/3) High deliveries — prevents clustering of urgent items

Solver settings: gapRel=0.05 stops when within 5% of optimum — no benefit searching further since the target is under 5% imbalance. timeLimit=10 provides a hard ceiling. Runtime: under 6 seconds across all test inputs.

Fallback: If solver fails, system switches to greedy and logs a warning. A safety net catches any unassigned deliveries left by floating point rounding and assigns them greedily. Output is always produced.

Task 5 — Build Output and Sequence Routes

Within each agent's assigned deliveries: group by priority tier, apply nearest-neighbour TSP within each group starting from the stop closest to the warehouse, then concatenate High → Medium → Low.

Output CSV columns: Agent, Location_ID, Priority, Distance_km, Sequence, Cumulative_km. The final Cumulative_km per agent gives total distance travelled, satisfying Task 5 directly.

4. System Evaluation

File	Imbalance	Compliance	Status
input1.csv	2.36%	75.00%	PASS
input2.csv	0.00%	100.00%	PASS
input3.csv	0.00%	100.00%	PASS
input4.csv	9.84%	100.00%	REVIEW
input5.csv	7.40%	100.00%	REVIEW
input6.csv	0.00%	75.00%	PASS
input7.csv	100.00%	100.00%	N/A
input8.csv	0.00%	100.00%	PASS
input9.csv	0.00%	100.00%	PASS
input10.csv	1.11%	100.00%	PASS

PASS (8/10): Six files achieved exactly 0% imbalance. The 75% compliance on input1 and input6 occurs when an agent has 4 deliveries with one High item the top-third window covers

2 slots, the High item lands at position 1 as intended, but the formula reads 75% due to route size. Both High deliveries are still completed before any Medium or Low stop. This is measurement sensitivity at small route sizes, not a sequencing failure.

REVIEW — input4: Distances range from 5km to 200km. The 200km Low-priority delivery forces a pairing that creates a 30km gap across agents. No assignment of these eight distances across three agents achieves sub-5% imbalance this is arithmetic, not an algorithm failure. Outlier detection flags it and recommends separate scheduling.

REVIEW — input5: Absolute gap is only 9km (125 vs 116km). The 7.40% reads high because the mean (121.67km) is small, amplifying the ratio. On larger datasets with 50+ deliveries this sensitivity disappears.

N/A — input7: Three deliveries for three agents. ILP is skipped, one delivery per agent assigned directly. Distances of 10km, 20km, 30km cannot be balanced across three agents. Correctly flagged as edge case.

5. Business Value

Workload fairness: Sub-5% imbalance across 8/10 files means agents' travel costs are distributed almost equally. An agent travelling 20% more than another for the same pay is a direct operational cost.

SLA protection: High-priority deliveries are not just assigned they are guaranteed to be executed first within each route through the hard sequencing rule in the route builder. Assignment without sequencing control is not an SLA guarantee.

Outlier detection: Deliveries more than 2 standard deviations above the mean distance are flagged before dispatch with a recommendation to schedule separately. This surfaces risk the operations manager needs to act on, rather than discovering it mid-route.

Reproducibility: Same input always produces same output. Auditable and defensible in any review.

6. Limitations

Distance as effort proxy: System optimises on km, not time. Real effort depends on traffic, road type, and stops. A production system would replace distance with travel time from a routing API. This is a data availability constraint.

TSP approximation: Nearest-neighbour uses warehouse distance difference as a proxy for inter-location travel distance. Without coordinates, a real distance matrix cannot be built. Produces sensible sequencing in practice but not a guaranteed shortest route.

Stop count imbalance: Count cap constraint limits lopsidedness but one long delivery and ten short ones can have identical km totals with very different workdays. A complete model would optimise on time per delivery, not distance.

Priority Compliance at small route sizes: Top-third formula is coarse for 2–3 stop routes. A High item at position 1 in a two-stop route still reads as partial compliance when both stops are High. Resolves naturally on routes with 7+ stops per agent.

7. Conclusion

The system went from a basic sort-and-assign script to a multi-layer optimisation pipeline with measurable quality guarantees, operational risk detection, and batch capability. Each version addressed a specific gap.